FOCES GROUP — INTERNAL BUILD DOCUMENT

PROMPT PARADOX 2.0 TECHNICAL SPEC

OVERMINDEDITION

Complete website build instructions, UI guidelines, architecture decisions, and scalability requirements.

MAX PARTICIPANTS	80 simultaneous
JUDGING	7 auto-evaluated + 1 manual (Level 5)
PLATFORM	Web (desktop-first, mobile-supported)
AUTH	Individual login with name + college

ARCHITECTURE

Tech Stack & System Design

RECOMMENDED STACK

Frontend: React + Vite + Tailwind CSS
Backend: Firebase (Firestore + Auth + Storage + Hosting)
Admin Panel: Separate React route, protected by Firebase Auth role
Image Hosting: Static files in Firebase Hosting /public (not Cloudinary — CDN recompression will destroy steganography data in Level 2)
Screenshot uploads (Level 5): Firebase Storage
Leaderboard: Firestore, polled every 60 seconds (not live listeners — see scalability section)

WHY FIREBASE FOR 80 CONCURRENT USERS

Firebase Firestore handles thousands of concurrent connections without configuration. For 80 simultaneous participants, it is well within the free tier limits. Firestore's document model maps perfectly to the participant state structure (one document per participant, updated on each level completion). Firebase Hosting serves static assets from a global CDN — no server load for page renders.

PARTICIPANT DATA STRUCTURE (Firestore)

```
Collection: participants
Document ID: uid (Firebase Auth UID)

Fields:
name: string
college: string
currentLevel: number (1–8)
completedLevels: array of numbers
levelTimestamps: map { '1': timestamp, '2': timestamp ... }
attemptCounts: map { '1': number, '2': number ... }
hintsUsed: array of numbers
startTime: timestamp
finishTime: timestamp (null until complete)
level5Status: string ('pending' | 'approved' | 'rejected')
```

ADMIN DATA STRUCTURE (Firestore)

```
Collection: level5Submissions
Document ID: auto-generated

Fields:
uid: string
name: string
college: string
prompt: string
screenshotUrl: string (Firebase Storage URL)
submittedAt: timestamp
status: string ('pending' | 'approved' | 'rejected')
```

```
reviewedAt: timestamp
```

UI DESIGN

OVERMIND Visual Identity & UX

OVERALL AESTHETIC

The website should feel like a living terminal — an interface that OVERMIND itself built. Dark, glitchy, intelligent, and slightly unsettling. Every screen should make the participant feel like they are inside something, not just visiting a website. Think: cyberpunk hacking interface crossed with a high-end game UI.

COLOUR PALETTE

Background	`#0A0A0F`	Near-black. Used on every page.
Card/Panel BG	`#13132A`	Slightly lighter. All content cards.
Primary Accent	`#7B2FFF`	Electric violet. Borders, level numbers, highlights.
Secondary Accent	`#00E5FF`	Cyan. OVERMIND's dialogue, code text.
Gold	`#FFD700`	Difficulty stars, important labels.
Success / Answer	`#00FF88`	Correct submission flash, unlocked indicators.
Warning / Error	`#FF3C5C`	Wrong answer, rate limit warnings.
Body Text	`#F0F0FF`	All readable content.
Muted Text	`#8888AA`	Hints, metadata, secondary info.

TYPOGRAPHY

- Primary font: **Space Mono** or **JetBrains Mono** (Google Fonts, free) — monospace, hacker aesthetic
- Headings: same font, bold, letter-spacing: 0.1em
- OVERMIND dialogue: italic, cyan (#00E5FF), slightly smaller
- All fonts loaded via Google Fonts CDN — no local files needed

CSS EFFECTS (implement these — they define the feel)

Glitch text animation

CSS keyframe animation that briefly offsets a pseudo-element in red/cyan on heading text. Apply to OVERMIND's name and level titles. Triggers on page load and on level unlock. ~15 lines of CSS.

Scanline overlay

A fixed ::after pseudo-element on the body with repeating-linear-gradient of semi-transparent black lines. Gives a CRT monitor feel. Opacity: 0.03 — subtle, not distracting.

Cursor blink on terminals

Level 1 and any code-display areas use a blinking block cursor (| or ■) via CSS animation. Makes it feel like a live terminal.

Card flip animation (Level 3)

CSS perspective + rotateY transform on hover/tap. The front face shows a dark glitch pattern. The back face reveals the QR code. transition: 0.4s ease.

Level unlock flash

On correct answer submission, a full-screen overlay flashes green (#00FF88 at 30% opacity) for 300ms, then fades. Followed by OVERMIND's transition message appearing with a typewriter animation before the next level loads.

Typewriter text

OVERMIND's dialogue at each level types itself out character by character at ~40ms per character using a JavaScript setInterval loop. Gives the impression that OVERMIND is speaking to you in real time.

SCREENS

Page-by-Page Breakdown

REGISTRATION PAGE

Fields: Full Name, College/Institution, Email (used as login).

On submit: Firebase Auth creates account with email + auto-generated password sent to their email. Firestore document created with currentLevel: 1.

Design: Full-screen dark background. Centred form card with violet border. OVERMIND's opening monologue plays as typewriter text above the form. No logo — just the event name in glitch font at the top.

Validation: Unique email check, all fields required.

LOBBY / WAITING SCREEN

Shown after registration until the organizer flips the 'event started' flag in Firestore. Displays: participant name, a countdown timer if a start time is set, and a live count of how many participants have registered. OVERMIND message: 'Initialising trials. Stand by, [name].' Background shows slowly scrolling binary code (CSS animation, no canvas needed).

LEVEL SCREEN (core game screen)

Layout: Top bar shows level number, participant name, elapsed time, hints remaining. Centre: OVERMIND's dialogue (typewriter), then challenge content below it. Bottom: answer input field + submit button.

Level-specific content areas (see Level Details section below).

Wrong answer: input field flashes red, error message from OVERMIND appears. Correct answer: green flash, OVERMIND transition message, next level loads after 3s.

LEADERBOARD PAGE

Accessible during the event from a nav link (does not interrupt current level). Shows: rank, participant name, college, current level reached, time elapsed. Sorted by: levels completed DESC, then time ASC. Updates every 60 seconds via Firestore polling (not live listeners — prevents excessive reads under load). Top 3 entries highlighted with gold/silver/bronze accent colours. Badge shown next to names who used hints ('H' indicator).

WIN SCREEN

Triggered when the first participant submits the correct Level 8 answer. Full-screen takeover: OVERMIND's final monologue types out. Winner's name displayed in large glitch text. For all other participants still playing: a non-intrusive banner appears at the top of their screen saying '[Name] has been chosen. The trials continue.' They can still complete all levels — final leaderboard shows full rankings.

ADMIN PANEL (organizer-only route)

Route: /admin — protected by Firebase Auth custom claim (role: admin). Features: Live list of all Level 5 submissions with screenshot preview, participant name, and their pasted prompt. Approve/Reject buttons. On approve: Firestore updates participant's currentLevel to 6 automatically. On reject: participant notified on their screen with resubmission option. Also shows: live participant count per level, event start/stop toggle.

LEVEL UI DETAILS

Per-Level Frontend Implementation

LEVEL 1 — The Handshake

Full-screen fake terminal. Black background with green/cyan monospace text. Blinking cursor after the prompt. Input field styled as a terminal input line (no border box, just an underline or cursor). Submit on Enter key, not just button click. Display the binary question text in a typewriter sequence.

LEVEL 2 — Pixel Confession

Image displayed centred, roughly 400px wide. Slight CSS flicker animation on the image (opacity oscillation between 0.95 and 1.0) to suggest it is a 'transmission.' Download button below the image. Standard text input for the passphrase answer.

LEVEL 3 — The Glitch Gallery

3x3 CSS grid of cards, each 120x120px. Cards have a dark face with a noise/static CSS background pattern. On hover (desktop) or tap (mobile), card flips via CSS rotateY transform revealing the QR code on the back face. CRITICAL: QR images must be lazy-loaded server-side on hover — do NOT embed all 9 image URLs in the initial page HTML or they can be extracted from DevTools. Use a Firebase Cloud Function that returns the image URL only when a specific card index is requested, rate-limited to one request per card per session.

LEVEL 4 — The Dead Drop

Clean text layout. The clue text displayed in styled paragraph format with the key phrase 'RFC 1' subtly highlighted in cyan on second read (not immediately, use a 3-second delay before the highlight appears — reward careful reading). Standard text input, case-insensitive evaluation.

LEVEL 5 — The Prompt Architect

Two-column layout on desktop, stacked on mobile. Left: target output in a styled 'transmission box' with cyan border. Banned words listed below in red. Right: file upload area for screenshot (drag-and-drop + click to upload, accept image/* only, max 5MB) and a textarea for their prompt text. On submit: files upload to Firebase Storage, submission document created in Firestore, participant sees 'Awaiting OVERMIND approval...' with a pulsing indicator. Firestore listener on their submission document — when admin approves, screen auto-advances without refresh.

LEVEL 6 — Logic Bomb

Defusal aesthetic: dark panel with a red countdown visual (purely decorative, does not actually count down). Five toggle switches labelled A–E, styled as large, satisfying ON/OFF switches with CSS. Default state: all FALSE. A 'DEFUSE' button at the bottom. On wrong combination: a red 'DETONATION PREVENTED — SYSTEM LOCKED' message with the attempt count shown. Rate limit: 3 wrong attempts per 2 minutes stored in Firestore.

LEVEL 7 — The Mirror

Visually identical to previous level screens. The word OVERMIND is written in white text on a white background div, placed within the body content. Flavour line: 'Everything you need is already here. It always was.' No other visual hints. Standard text input.

LEVEL 8 — Paradox Protocol

The ciphertext displayed in a styled 'intercepted transmission' box — monospace, cyan text, with a faint border glow animation. Instruction text above: 'OVERMIND's final transmission has been intercepted and encoded. Decode it to find the passphrase.' Standard text input. Accept SIGNAL_FOUND case-insensitively, with or without underscore replaced by space.

SCALABILITY & ANTI-CHEAT

Handling 80 Simultaneous Participants

WHY 80 USERS WILL NOT CAUSE PROBLEMS

Firebase Firestore's free tier supports 50,000 reads and 20,000 writes per day. 80 participants each writing ~10 level completions = 800 writes total. Leaderboard polled every 60s by 80 users over a 2-hour event = ~9,600 reads. Both are well within free tier. The only potential issue is live Firestore listeners on the leaderboard — replace with polling (setInterval every 60s) to cap reads.

LEADERBOARD — USE POLLING, NOT LIVE LISTENERS

Do NOT use Firestore onSnapshot for the leaderboard. Use a simple fetch call every 60 seconds. onSnapshot holds an open WebSocket connection per client — 80 simultaneous connections is fine technically, but polling is more predictable and avoids any connection limit concerns on the free plan.

LEVEL 5 SCREENSHOT UPLOADS — SIZE LIMIT

Enforce a 5MB max on screenshot uploads client-side before the file reaches Firebase Storage. Most screenshots are under 500KB. Add a client-side check with a clear error message. Firebase Storage free tier is 5GB total — 80 submissions at 5MB max = 400MB absolute worst case, well within limits.

RATE LIMITING ANSWER SUBMISSIONS

Store attemptCounts per level per participant in their Firestore document. Before processing any answer, read this value. If >= 3 attempts in the last 2 minutes (compare against lastAttemptTime field), reject with a cooldown message. This logic runs client-side with Firestore validation rules as backup — do not rely on client-side alone.

FIRESTORE SECURITY RULES — CRITICAL

Participants should only be able to read/write their own document. Rule: allow read, write: if request.auth.uid == resource.data.uid; The leaderboard collection is read-only for all authenticated users. The level5Submissions collection is write-only for participants, read/write for admins only. Without these rules, participants can read each other's answers from Firestore directly.

LEVEL 3 QR LAZY LOADING — ANTI-CHEAT

This is the most important anti-cheat measure in the entire build. Do not embed QR image URLs in the page HTML. Create a Firebase Cloud Function endpoint: /getCardImage?index=N This function checks that the participant is authenticated, then returns the image URL for card N. Rate limit: one request per card index per session. This prevents DevTools extraction of all 9 URLs at once.

ANSWER CASE NORMALISATION

Normalise all answer inputs server-side before comparison: trim whitespace, convert to lowercase (or uppercase consistently), strip extra spaces. Binary answers: accept with or without spaces between bytes. This prevents participants from getting stuck on trivial formatting issues and reduces support overhead during the event.

EVENT START/STOP CONTROL

Add an 'eventActive' boolean document in Firestore. All level submission endpoints check this flag before processing. If false, all submissions are rejected. This lets organizers start the event simultaneously for all 80 participants with one Firestore write, and stop it cleanly at the end.

BUILD PLAN

Implementation Sequence & Timeline

PHASE 1 — FOUNDATION (Day 1–2)

- ☐ Set up Firebase project: enable Auth, Firestore, Storage, Hosting
- ☐ Create React + Vite project, configure Tailwind with OVERMIND colour tokens
- ☐ Implement Firebase Auth: registration flow (name, college, email)
- ☐ Create Firestore participant document on registration
- ☐ Build the base Level screen component (reusable shell for all 8 levels)
- ☐ Build the top navigation bar (level indicator, timer, hints remaining)
- ☐ Set up Firestore security rules

PHASE 2 — LEVELS 1–4 (Day 3–4)

- ☐ Level 1: Terminal UI component, binary answer validation
- ☐ Level 2: Image display with download, passphrase input
- ☐ Level 3: Card flip grid, Firebase Cloud Function for lazy QR loading, rate limiting
- ☐ Level 4: Clue text layout with delayed highlight, case-insensitive answer check
- ☐ Implement hint system: hint content per level, Firestore deduction, UI display
- ☐ Implement rate limiting (3 attempts per 2 minutes) across all levels

PHASE 3 — LEVELS 5–8 + ADMIN (Day 5–6)

- ☐ Level 5: File upload UI, Firebase Storage integration, Firestore submission doc, pending screen
- ☐ Admin panel: submission review UI, approve/reject buttons, auto-unlock on approve
- ☐ Level 6: Toggle switch UI, Boolean logic evaluation, defusal aesthetic
- ☐ Level 7: Hidden text implementation, standard answer input
- ☐ Level 8: Ciphertext display box, ROT-13 answer validation
- ☐ Win screen: full-screen takeover, winner announcement banner for other participants

PHASE 4 — LEADERBOARD + POLISH (Day 7)

- ☐ Leaderboard page: Firestore polling (60s), ranking logic, badges
- ☐ Lobby/waiting screen with event start toggle
- ☐ CSS effects: glitch animation, scanline overlay, typewriter text, card flip
- ☐ OVERMIND dialogue content: write all 8 level messages + transition messages
- ☐ Event start/stop control in admin panel
- ☐ Mobile responsiveness check on all screens

PHASE 5 — TESTING (Day 8–9)

- ☐ Full run-through: register as test user, complete all 8 levels end-to-end
- ☐ Test with 3–5 simultaneous test accounts to verify Firestore concurrency
- ☐ Test Level 3 QR lazy loading — verify DevTools cannot extract all URLs
- ☐ Test Level 5 admin flow: submit screenshot, approve, verify Level 6 unlocks
- ☐ Test rate limiting: submit wrong answers rapidly, verify lockout triggers
- ☐ Verify steg image survives Firebase Hosting (do not upload via Storage)
- ☐ Test on mobile: all levels, especially Level 3 tap-to-flip
- ☐ Verify CERN page answer (visit RFC 1, confirm 'Network' is still the first word)

PHASE 6 — PRE-EVENT (Day 10)

- ☐ Deploy to Firebase Hosting
- ☐ Prepare steg image (encode VOIDWALKER), test decode on both recommended tools
- ☐ Prepare Level 3 QR code linking to Google Drive image
- ☐ Set up admin account with custom claim
- ☐ Brief organizer team on Level 5 review process and admin panel usage
- ☐ Set eventActive: false in Firestore (event does not start until organizers flip it)
- ☐ Send registration link to participants

EFFORT SUMMARY

Timeline at a Glance

PHASE	TASK	DAYS
1	Firebase setup, Auth, base components	2
2	Levels 1–4 + hint system + rate limiting	2
3	Levels 5–8 + admin panel	2
4	Leaderboard + CSS effects + polish	1
5	End-to-end testing + concurrency testing	2
6	Pre-event prep, deploy, briefing	1
TOTAL		~10 days

DOCUMENT END

Prompt Paradox 2.0 — Technical Build Specification

This document is intended for the FOCES development team. All implementation decisions, colour values, Firestore structures, and security rules described here are final unless discussed with the FOCES organizers.